

Differentiable Programming for Audio

Differentiable Physical Modelling and Machine Learning for Sound Synthesis

Rodrigo Diaz

2026-05-28

Week 4 – Differentiable Programming for Audio

1

Overview

- Recap: physical models as differentiable audio synthesisers
- The Week 4 turnaround: from synthesis to learning
- Differentiable programming in JAX
- Gradients through loops, recursions, and simulators
- Audio losses: waveform, spectrum, and perceptual tradeoffs
- Mini task: fit a small differentiable audio model
- Practical problems: constraints, stability, and local minima

Recap: Physical Models as Programs

What We Have Built So Far

physical parameters + excitation $\longrightarrow$ simulator $\longrightarrow$ sound

- Week 1: physical models as state-space systems
- Week 2: modal and grid discretisations
- Week 3: nonlinear forces and coupled systems

The common thread:

$$\theta \longrightarrow \hat{y}$$

Parameters go in. Audio comes out.

The One Change in Week 3

Linear model:

inertia + damping + linear restoring force = external force

Week 3 added:

nonlinear force $f_{\text{nl}}(\mathbf{q})$

So the model became:

linear dynamics + $f_{\text{nl}}(\mathbf{q})$ = external force

For tension modulation:

$$f_{\text{nl},\mu}(\mathbf{q}) \propto \lambda_{\mu}^2 q_{\mu} \sum_{\nu} \lambda_{\nu}^2 q_{\nu}^2$$

The force now depends on the current (modal) state of the system.

Week 4 – Differentiable Programming for Audio

The Week 4 Turnaround

So far:

$$\theta \longrightarrow \hat{y}$$

This week:

$$\ell(\hat{y}, y) \longrightarrow J(\theta) = \ell(g(\theta), y) \longrightarrow \nabla_{\theta} J \longrightarrow \text{update } \theta$$

The question changes:

Can we simulate this sound? $\longrightarrow$ Can we learn the model from sound?

Differentiable Programming for Audio

A Differentiable Audio Synthesiser

Write the synthesiser as a function:

$$\hat{y} = g(\theta)$$

Choose a scalar signal-level loss that compares audio:

$$\ell(\hat{y}, y)$$

Composing it with the synthesiser gives a scalar objective as a function of parameters:

$$J(\theta) = \ell(g(\theta), y)$$

Then use automatic differentiation. For each parameter θ_i , summing over audio samples n :

$$\frac{\partial J}{\partial \theta_i} = \sum_n \frac{\partial \ell}{\partial \hat{y}_n} \frac{\partial \hat{y}_n}{\partial \theta_i}$$

For a sinusoid, θ_i could be A , ω , f , or ϕ .

A Tiny Differentiable Synthesiser

Start with one damped sinusoid:

$$\hat{y}_n = Ae^{-\gamma t_n} \sin(2\pi f t_n + \varphi)$$

The learnable parameters are:

$$\theta = \{A, \gamma, f, \varphi\}$$

Notebook: [week04/notebooks/differentiable_synthesis.py](https://github.com/philipphusmann/differentiable_synthesis/blob/master/week04/notebooks/differentiable_synthesis.py)

This revisits the Week 2 optional challenge: optimise the argument of `sin`.
Even this simple synthesiser is already a differentiable audio synthesiser.

Fit It to a Target

Define a waveform loss between signals:

$$\ell_{\text{MSE}}(\hat{y}, y) = \frac{1}{N} \sum_n (\hat{y}_n - y_n)^2$$

For this synthesiser, the optimisation objective is:

$$J(\theta) = \ell_{\text{MSE}}(g(\theta), y)$$

Then update parameters using gradients:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J$$

Here η is the learning rate: the step size for each update.

Automatic differentiation gives the gradient: local direction and magnitude of sensitivity.

Same Synth, Complex Recursion

The same oscillator dynamics can be advanced as a complex state:

$$z^{n+1} = az^n$$

with pole and initial state

$$a = e^{(-\gamma + i2\pi f)\Delta t}, \quad z^0 = Ae^{i(\varphi - \pi/2)}$$

and the audio output is:

$$\hat{y}_n = \operatorname{Re}\{z^n\}$$

This is the same complex multiplication pattern from Week 3, now inside a loss function.

Optimising One Frequency



Differentiable Does Not Mean Easy

- Frequency errors accumulate as **phase errors**.
- Waveform MSE creates many local minima.
- Initialisation changes the optimisation outcome.
- The loss function decides what “close” means.

Automatic differentiation gives derivatives of the program we wrote, up to floating-point arithmetic.

It does not guarantee that the optimisation problem is easy.

Keeping Poles Stable

The discrete recursion is stable when:

$$z^{n+1} = az^n, \quad |a| < 1$$

Instead of learning a directly, learn an unconstrained complex parameter:

$$p \in \mathbb{C}$$

and squash it into the unit disk:

$$a = h(p) = \begin{cases} \frac{\tanh |p|}{|p|} p, & p \neq 0, \\ 0, & p = 0. \end{cases}$$

Then:

$$|a| = \tanh |p| < 1$$

Continuous vs. Discrete Parameters

Continuous-time view:

$$a = e^{(-\gamma + i2\pi f)\Delta t}, \quad \gamma \geq 0$$

Discrete-time view:

$$a = h(p), \quad |a| < 1$$

If needed, recover the derived continuous-time modal parameters:

$$\gamma = -\frac{\log |a|}{\Delta t}, \quad f = \frac{\angle a}{2\pi\Delta t}$$

From a discrete pole a , we can recover a modal frequency, but only up to discrete-time aliasing.

These are modal quantities derived from the physical model, not the physical parameters themselves.

The JAX Pattern

```
1 def synth(theta):  
2     return y_hat  
3  
4  
5 def loss(theta):  
6     y_hat = synth(theta)  
7     return jnp.mean((y_hat - y_target) ** 2)  
8  
9  
10 loss_value, grads = jax.value_and_grad(loss)(theta)  
11  
12 updates, opt_state = optimizer.update(grads, opt_state, theta)  
13 theta = optax.apply_updates(theta, updates)
```

Once the synthesiser is a JAX function, the training loop looks like any other differentiable model.

Audio Losses

The Loss Is Part of the Model

The training loop contains two choices:

$$\theta \longrightarrow \hat{y} \longrightarrow \ell(\hat{y}, y)$$

This defines the parameter objective $J(\theta) = \ell(g(\theta), y)$.

- the synthesiser decides what sounds are possible
- the loss decides what counts as “close”
- the gradient follows the loss, not our intention

So changing the loss can change the optimisation problem as much as changing the model.

Waveform Losses

The simplest losses compare samples.

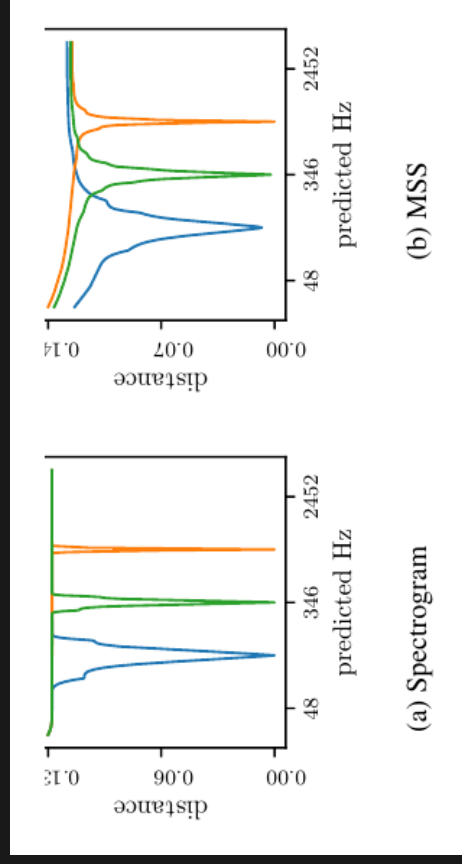
MSE is the mean squared, or normalized squared L2, waveform loss:

$$\ell_{\text{MSE}} = \frac{1}{N} \sum_n (\hat{y}_n - y_n)^2$$

$$\ell_{\text{L1}} = \frac{1}{N} \sum_n |\hat{y}_n - y_n|$$

- direct and easy to differentiate
- L1 is less dominated by large sample errors than MSE
- works well when the target is phase-aligned
- very sensitive to small frequency and **phase errors**
- creates the kind of local structure we just saw

Spectral Loss



Turian and Henry, 2020.

Compare short-time magnitude spectra:

$$Y = \text{STFT}(y), \quad \hat{Y} = \text{STFT}(\hat{y})$$

t indexes time frames; k indexes frequency bins.

$$\ell_{\text{mag}} = \frac{1}{N_t N_f} \sum_{t,k} \left(|\hat{Y}_{t,k}| - |Y_{t,k}| \right)^2$$

This is less tied to sample alignment, but it still compares bins **coordinate by coordinate**.

For pitch, the gradient can be weak or misleading away from the target.

Turian and Henry, “I’m Sorry for Your Loss: Spectrally-Based Audio Distances Are Bad at Pitch”, 2020.

Log Magnitude

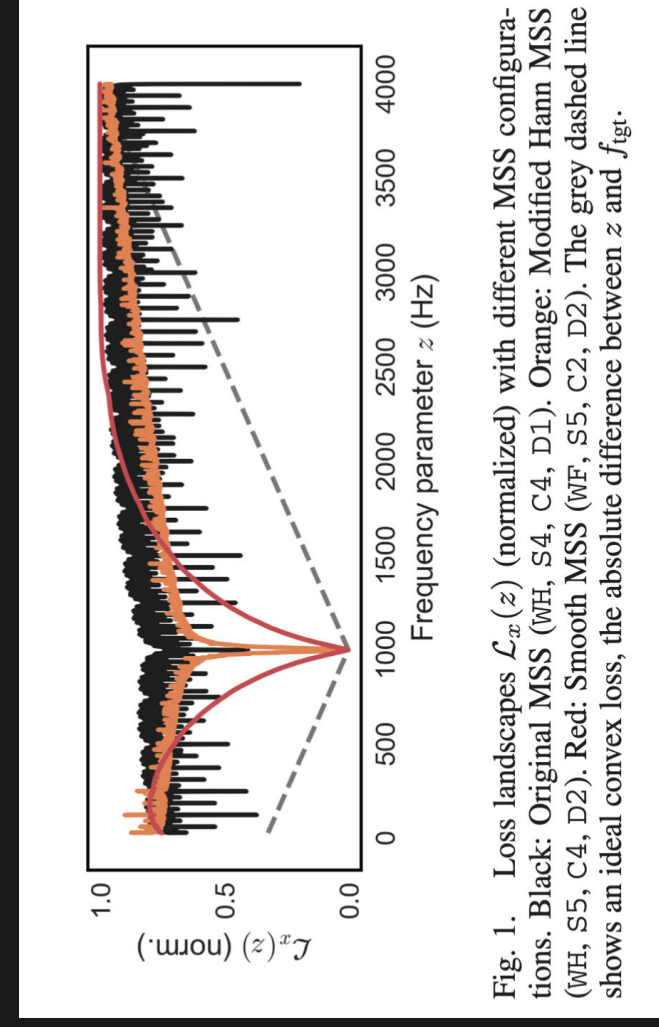
Raw magnitude is dominated by the loudest bins.

Log magnitude compares spectral shape on a compressed scale:

$$\ell_{\text{log-mag}} = \frac{1}{N_t N_f} \sum_{t,k} \left(\log(\epsilon + |\hat{Y}_{t,k}|) - \log(\epsilon + |Y_{t,k}|) \right)^2$$

Quieter components now have more influence on the gradient.

Multi-Resolution STFT Loss



Schwär and Müller, 2023.

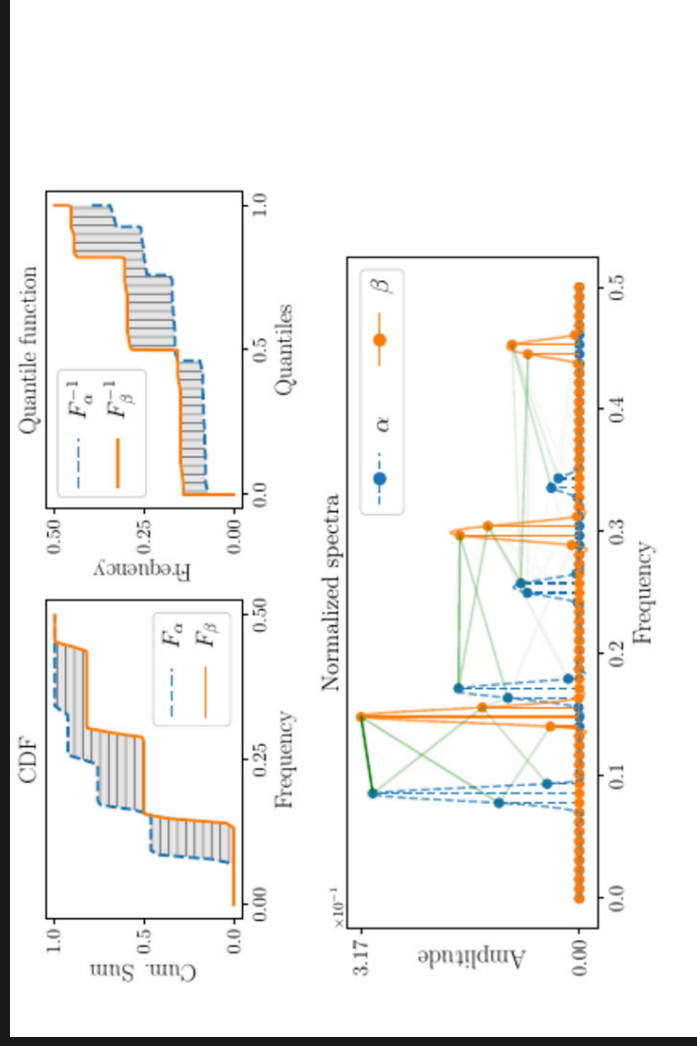
See Schwär and Müller, “Multi-Scale Spectral Loss Revisited”, IEEE Signal Processing Letters, 2023.

Use several STFT settings:

$$\ell_{\text{MR-STFT}} = \sum_m \ell_{\text{STFT}}^{(m)}$$

- short windows: attacks and local timing
- long windows: pitch and harmonic structure
- different hop sizes and windows give different gradients

Spectral Optimal Transport Loss



Torres, Peeters, and Richard, 2024.

Torres, Peeters, and Richard, “Unsupervised Harmonic Parameter Estimation Using Differentiable DSP and Spectral Optimal Transport”, ICASSP 2024.

Instead of comparing each frequency bin independently:

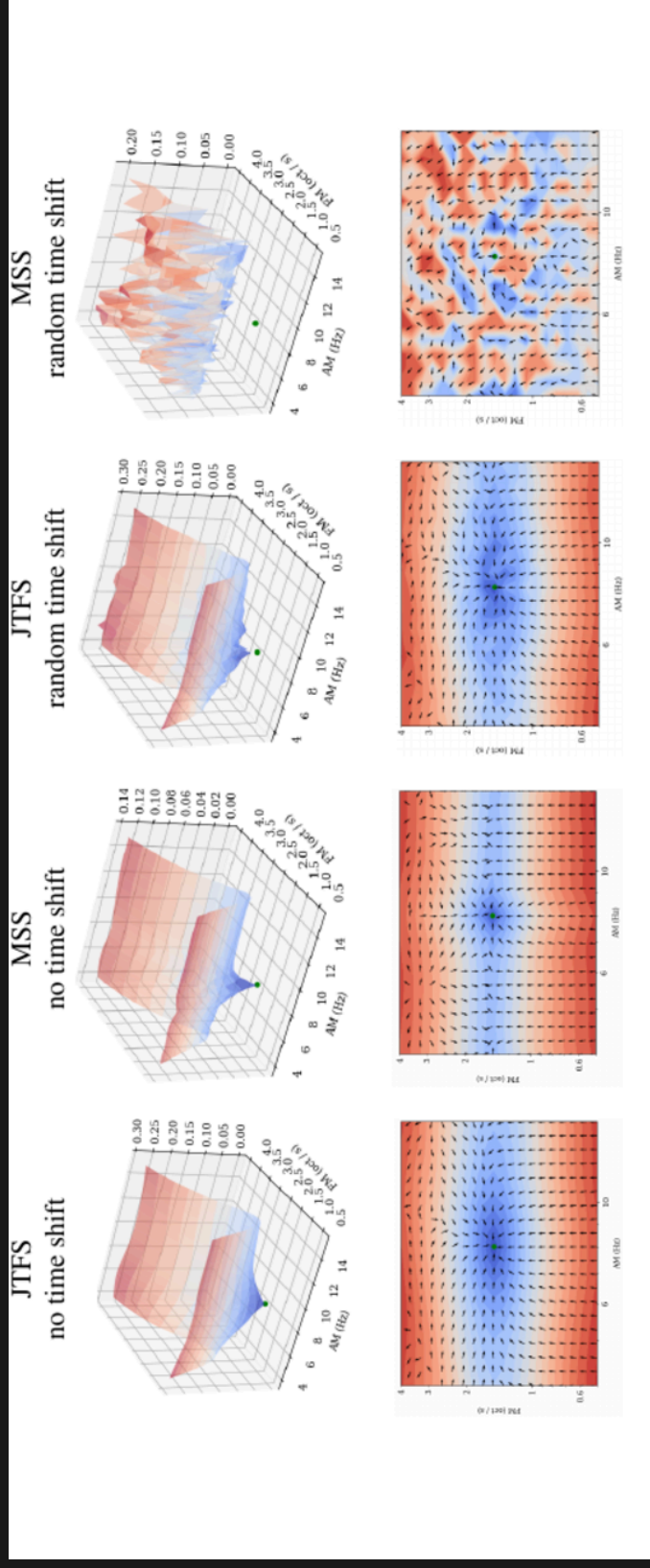
$$\ell_{\text{SOT}} = \sum_t W_p(\tilde{S}_{\hat{y},t}, \tilde{S}_{y,t})$$

Use normalized nonnegative spectral frames $\tilde{S}_{\cdot,t}$.

The loss measures how far spectral energy moves.

It can give useful gradients for pitch and harmonic displacement.

JTFS / Scattering Loss



Vahidi et al., 2023.

$$\ell_{\text{JTFS}} = \|\Phi_{\text{JTFS}}(\hat{y}) - \Phi_{\text{JTFS}}(y)\|_2^2$$

JTFS compares spectrotemporal scattering features: texture, articulation, and mesostructure rather than raw STFT bins.

Vahidi et al., “Mesostructures: Beyond Spectrogram Loss in Differentiable Time-Frequency Analysis”, 2023. See also Muradeli et al., “Differentiable Time-Frequency Scattering on ~~Week 4~~ Differentiable Programming for Audio

Loss Takeaway

There is no neutral audio loss.

- waveform loss is precise but phase-sensitive
- spectral magnitude losses are more tolerant but can still mislead pitch optimisation
- multi-resolution STFT losses combine several time-frequency views
- the loss and the parameterisation together shape the gradient landscape

Next: use these losses to fit physical or modal model parameters.

Learning Modal Parameters

From One Oscillator to a Modal Synth

The one-oscillator example is the smallest modal synthesiser.

For a modal bank:

$$z_{\mu}^{n+1} = a_{\mu} z_{\mu}^n$$

with:

$$a_{\mu} = e^{(-\gamma_{\mu} + i2\pi f_{\mu})\Delta t}$$

The output is a readout over modes:

[Math Processing Error]

Same idea, just more poles.

What Can We Learn?

The parameter vector might contain:

$$\theta = \{f_\mu, \gamma_\mu, c_\mu, z_\mu^0\}$$

- modal frequencies
- modal damping rates
- readout gains
- initial modal amplitudes and phases
- excitation parameters
- nonlinear force parameters

The modelling question becomes:

Which parameters should remain physical, and which should be learned freely

Vectorised Complex Recursion

For all modes at once:

$$\mathbf{z}^{n+1} = \mathbf{a} \odot \mathbf{z}^n$$

```

1 def modal_step(z, _):
2     y = jnp.real(jnp.sum(c * z))
3     z_next = a * z
4     return z_next, y
5
6
7 _, y_hat = jax.lax.scan(
8     modal_step,
9     init=z0,
10    xs=None,
11    length=n_steps,
12 )

```

`lax.scan` is just a loop, but JAX can differentiate through it.

Differentiating Through Time

The loss depends on every output sample:

[Math Processing Error]

Each output depends on all previous updates:

$$z^n = a^n z^0$$

For a real parameter α such as f or γ , the gradient accumulates through time:

$$\frac{\partial J}{\partial \alpha} = \sum_n \frac{\partial \ell}{\partial \hat{y}_n} \frac{\partial \hat{y}_n}{\partial z^n} \frac{\partial z^n}{\partial \alpha}$$

This is backpropagation through the simulator.

When Gradients Get Too Long

Backpropagation through a simulator is still backpropagation through a long recurrent computation.

For a damped modal pole:

$$a_\mu = e^{(-\gamma_\mu + i2\pi f_\mu)\Delta t}$$

the sensitivity to the initial state scales like:

$$\frac{\partial z_\mu^n}{\partial z_\mu^0} = a_\mu^n, \quad |a_\mu|^n = e^{-\gamma_\mu n \Delta t}$$

- long damped sequences can kill late gradients
- unstable recursions can explode gradients
- full BPTT stores the whole unrolled simulation
- truncated BPTT is cheaper, but biased toward short dependencies

See Sutskever, [Training Recurrent Neural Networks](#), arXiv:1410.1199, 2013.

Week 4: Differentiable Programming for Audio

A Practical Split

Start small:

- learn frequencies and damping from a clean synthetic target
- keep the number of modes fixed
- keep excitation and readout simple
- constrain poles so the recursion stays stable

Then increase difficulty:

- unknown excitation
- noisy or recorded target audio
- too many or too few modes
- nonlinear forces

Mini Task: Fit a Differentiable Synth

Notebook: [week04/notebooks/differentiable_synth.py](https://github.com/robertknight/differentiable_synth/blob/main/week04/notebooks/differentiable_synth.py)

Start with the small case:

$$\theta = \{A, \gamma, f, \varphi\}$$

Then inspect the complex-recursion version:

$$z^{n+1} = az^n$$

Try:

- changing the initial frequency
- changing the signal duration
- switching from waveform loss to spectral loss
- fitting one pole radius directly instead of fitting γ

Preview: Physical Parameters

From Modal to Physical Parameters

Instead of learning each modal frequency independently:

$$\theta = \{f_\mu, \gamma_\mu\}_{\mu=1}^M$$

learn a smaller physical set, assuming ρ and A_{cs} are known:

$$\theta = \{T_0, EI, d_1, d_3\}$$

The physical parameters generate the modal parameters.

For a stiff string:

$$\omega_\mu^2 = \frac{T_0}{\rho A_{cs}} \lambda_\mu^2 + \frac{EI}{\rho A_{cs}} \lambda_\mu^4$$

For damping:

$$2\gamma_\mu = d_1 + d_3 \lambda_\mu^2$$

So tension, stiffness, and damping are not isolated modal knobs. They shape whole frequency and decay-rate curves.

Point Estimates, Not Uncertainty

This week we mostly do point estimation:

$$\hat{\theta} = \arg \min_{\theta} J(\theta), \quad J(\theta) = \ell(g(\theta), y)$$

The optimiser returns one best-fit parameter vector:

$$\hat{\theta}$$

This is not the same as estimating a posterior distribution:

$$\underbrace{\hat{\theta}}_{\text{point estimate}} \quad \text{is not} \quad \underbrace{p(\theta \mid y)}_{\text{posterior distribution}}$$

Several parameter settings may explain the same sound nearly equally well. Uncertainty and distribution estimation are for Week 9.

Maximum Likelihood View

Assume the target was generated by:

$$y = g(\theta) + \epsilon$$

with independent Gaussian noise:

$$\epsilon_n \sim \mathcal{N}(0, \sigma^2)$$

Then maximum likelihood gives:

$$\hat{\theta}_{\text{MLE}} = \arg \min_{\theta} \sum_n (y_n - \hat{y}_n(\theta))^2$$

So minimising waveform MSE is equivalent to MLE under a simple noise model. Other audio losses can be useful objectives, but they are not automatically simple likelihoods.

Wrapping Up

The Week 4 Loop

This week turned synthesis into estimation:

$$\theta \rightarrow \hat{y} \quad \text{became} \quad y \rightarrow \hat{\theta}$$

The practical loop is:

model $\rightarrow$ loss $\rightarrow$ gradient $\rightarrow$ update $\rightarrow$ checks

- constrain parameters so the simulator remains stable
- choose a loss that matches the task: perception, parameters, or both
- initialise from signal knowledge when possible
- inspect the sound, spectrum, residual, and fitted parameters

What the Gradient Gives You Gives Does Not Give

- local sensitivity of the loss
- the correct physical model
- credit assignment through the simulator
- a globally optimal solution
- a direction for parameter updates
- uncertainty about the answer

Use the loss as a diagnostic: it can support an approximate fit, or expose a misleading landscape.

$$g(\hat{\theta}) \approx y \not\Rightarrow \hat{\theta} = \theta_{\text{true}}$$

Takeaway

differentiable audio = differentiable audio synthesiser + loss + constraints

- The loss defines what “close” means.
- Parameterisation shapes what the optimiser can safely change.
- AD gradients are derivatives of the implemented program, not magic for the inverse problem.
- Good inverse modelling is part optimisation, part physics, and part listening.

Looking ahead: fit modal banks, physical parameters, excitation, nonlinear forces, and learned residuals.

Which parts should be physics, which parts should be learned, and how do we know?

Resources

Sinusoidal frequency estimation by gradient descent

Fast differentiable modal simulation of non-linear strings, membranes, and plates

A review of differentiable digital signal processing for music and speech synthesis

Differentiable audio synthesis / DDSP tutorial:

intro2ddsp.github.io